

ANSWERS — MWIT Interactive Web Workbook สัปดาห์ที่ 11

☁ Upgrade Your Web App: เปลี่ยนการบันทึกข้อมูลจาก Local Storage ไปสู่ Supabase Cloud Database

โปรเจกต์: FinGoal — วางแผนการเงิน (Mini Project) บทเรียนนี้ครอบคลุมเฉพาะ **Create + Read** (บันทึกด้วย `insert()` และโหลดด้วย `select()`) — ยังไม่รวม Update, Delete และ Authentication (อยู่ในบทถัดไป)

STEP 1 — 🧱 ออกแบบ Table

สำรวจเว็บเดิม: เว็บรับข้อมูลอะไรบ้าง?

เว็บ FinGoal รับข้อมูลจากผู้ใช้ 2 กลุ่มหลัก:

1. **รายการรายรับ — รายจ่าย (Transaction)** ผ่านฟอร์มบันทึกรายการ:
 - ประเภท (type): รายรับ หรือ รายจ่าย
 - หมวดหมู่ (category): เช่น เงินเดือน, อาหาร, ค่าเดินทาง
 - จำนวนเงิน (amount): บาท
 - รายละเอียด (note): เช่น “ค่าอาหารกลางวัน”
 - วันที่ (date)
2. **เป้าหมายการออม (Goal)** ผ่านฟอร์มตั้งเป้าหมาย:
 - ชื่อเป้าหมาย (name): เช่น “MacBook”, “เที่ยวญี่ปุ่น”
 - เงินเป้าหมาย (target): บาท
 - เงินออมปัจจุบัน (current): บาท
 - ระยะเวลาแบบ (mode): รายวัน / รายเดือน / รายปี
 - จำนวนระยะเวลา (duration)

ตอนนี้ Local Storage เก็บอะไร?

Key ใน localStorage	เก็บอะไร	รูปแบบ
fingoal_transactions	รายการรายรับ/รายจ่ายทั้งหมด	array ของ object
fingoal_goals	เป้าหมายการออมทั้งหมด	array ของ object
fingoal_mode	โหมดการออม (deduct / remain)	string
fingoal_savings_balance	ยอดเงินออมปัจจุบัน	number
fingoal_total_deducted	ยอดเงินที่หักออมสะสม	number

หนึ่ง row ในเว็บของคุณหมายถึงอะไร?

- ตาราง transactions: **1 row = 1 รายการรายรับหรือรายจ่าย** ที่ผู้ใช้บันทึกหนึ่งครั้ง (1 object ใน JavaScript กลายเป็น 1 row, property แต่ละตัวกลายเป็น column)
- ตาราง goals: **1 row = 1 เป้าหมายการออม**

Table ที่ออกแบบ

ตาราง transactions

Column	Type	หน้าที่	ตัวอย่าง
id	uuid (PK)	รหัสประจำรายการ	3f2c...
type	text	รายรับ / รายจ่าย	income
category	text	หมวดหมู่	อาหาร
amount	numeric	จำนวนเงิน (บาท)	150.00
note	text	รายละเอียด	ค่าอาหารกลางวัน
date	date	วันที่ทำรายการ	2026-08-05
created_at	timestampz	เวลาที่บันทึก (ใช้เรียงลำดับ)	2026-08-05 10:30:00+07

ตาราง goals

Column	Type	หน้าที่	ตัวอย่าง
id	uuid (PK)	รหัสเป้าหมาย	a91f...
name	text	ชื่อเป้าหมาย	MacBook
target	numeric	เงินเป้าหมาย (บาท)	50000.00
current	numeric	เงินออมปัจจุบัน	12000.00
mode	text	รายวัน/เดือน/ปี	daily
duration	integer	จำนวนระยะเวลา	30
created_at	timestampz	เวลาที่สร้าง	2026-08-05 10:30:00+07

หมายเหตุ: ข้อมูลที่ใช้เป็นข้อมูลทดลองเท่านั้น เนื่องจากตอนนี้ยังไม่มีระบบ Login ข้อมูลอาจมองเห็นร่วมกันได้ ตาม RLS policy ที่ครูกำหนด

STEP 2 — 🛠️ เชื่อมต่อ Supabase

เพิ่ม Supabase JavaScript Library

วางสคริปต์ก่อน </body> และก่อน script.js ของเว็บ:

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
<script src="script.js"></script>
```

สร้าง Client ใน script.js

```
const SUPABASE_URL = "https://yqwnlrgfbmrktuulmwon.supabase.co";
const SUPABASE_KEY = "sb_publishable_...";
```

```
const supabaseClient = supabase.createClient(SUPABASE_URL, SUPABASE_KEY);
```

สิ่งที่ได้เรียนรู้: - ใช้เฉพาะ **Publishable Key** (อนุญาตให้อยู่ในหน้าเว็บได้) — ห้ามนำ **Service Role Key** มาวางในเว็บเด็ดขาด เพราะมีสิทธิ์ข้าม RLS - สร้าง table ใน Supabase ให้เรียบร้อยก่อน แล้วใส่ URL + Publishable Key - ตรวจสอบผลลัพธ์: console ต้องไม่พบ error supabase is not defined (แปลว่าโหลด CDN เรียบร้อย)

STEP 3 — Upgrade Save: จาก setItem() เป็น insert()

ก่อน — Local Storage

```
localStorage.setItem("myData", JSON.stringify(data));
```

หลัง — Cloud (Supabase)

```
const { data, error } = await supabaseClient
  .from("transactions")
  .insert({ type, category, amount, note, date });
```

การอ่านค่าจาก input และการคำนวณเดิมยังใช้เหมือนเดิม — เปลี่ยนเฉพาะปลายทางที่เก็บข้อมูล

 Predict ก่อน Run: ถ้ากด Save สองครั้ง จะมีข้อมูลใน table ที่ row และเพราะเหตุใด?

คำตอบ: 2 rows

เพราะทุกครั้งที่เรียก insert() จะเป็นการ **เพิ่มรายการใหม่ (insert new row)** ไม่เหมือน localStorage.setItem() ที่เขียนทับค่าของ key เดิม ดังนั้นกด Save ที่ครั้ง = มี row เพิ่มที่ row

STEP 4 — Upgrade Load: อ่านหลาย rows ด้วย select()

โหลดและแสดงประวัติ

```
const { data, error } = await supabaseClient
  .from("transactions")
  .select("*")
  .order("created_at", { ascending: false });
```

```
// data = array ของ objects
// จากนั้นใช้ map() สร้าง HTML เพื่อแสดงผล
```

- select("*") → ขอข้อมูลทุก column
- order() → เรียงรายการใหม่ไว้ด้านบน
- data → array ของ objects
- map() → สร้าง HTML จากข้อมูลแต่ละ row

ทดลองข้าม Browser

1. บันทึกข้อมูลจาก Browser แรก (เช่น Chrome)
2. เปิดเว็บในอีก Browser หรืออุปกรณ์ (เช่น Firefox / มือถือ)
3. กด Load แล้วสังเกตผล

ผลต่างจาก Local Storage ที่สังเกตได้คืออะไร?

คำตอบ: ข้อมูลใน Browser ที่สอง **โผล่ขึ้นมาเหมือนกัน** เพราะข้อมูลถูกดึงจาก Supabase Cloud Database (server) **ตัวเดียวกัน** ไม่ได้อยู่ใน browser ใด browser หนึ่ง

ความต่างจาก Local Storage: - Local Storage เก็บข้อมูล **เฉพาะเครื่อง/browser ที่บันทึก** — เปิดอีกเครื่องจะไม่เห็น - Supabase เก็บข้อมูล **บน cloud server กลาง** — ทุกอุปกรณ์ที่เชื่อมต่อ table เดียวกันเห็นข้อมูลชุดเดียวกัน - ข้อมูลถูก **ซิงค์ผ่านอินเทอร์เน็ต** แบบ real-time (เมื่อโหลด/บันทึก)

ถ้ายังไม่สำเร็จ – การแก้ปัญหา

ปัญหา	สาเหตุ	วิธีแก้
relation ... does not exist	ชื่อ table ในโค้ดไม่ตรงกับชื่อ table ใน Supabase	ตรวจสอบชื่อ table ให้ตรงกัน
row-level security policy	ยังไม่มี policy อนุญาต Insert หรือ Select	เพิ่ม RLS policy ใน Supabase
column ... does not exist	ชื่อ property ไม่ตรงกับ column	ตรวจสอบชื่อ column ให้ตรงกัน
supabase is not defined	โหลด Supabase CDN ซ้ำ/ไม่ถูกต้อง	โหลด CDN ก่อน script.js

STEP 5 — Test Like a Developer & Reflection

Reflection: อธิบายสิ่งที่คุณเรียนรู้

บทเรียนนี้ทำให้ผมเข้าใจการย้ายระบบบันทึกข้อมูลจาก Local Storage (ข้อมูลอยู่ใน Browser) ไปสู่ Supabase Cloud Database (ข้อมูลอยู่ใน Cloud) สรุปสิ่งที่ได้เรียนรู้:

1. **การออกแบบ Table** — 1 object ใน JavaScript = 1 row ในฐานข้อมูล, property = column ต้องวิเคราะห์ก่อนว่าข้อมูลในเว็บมีโครงสร้างอะไรบ้าง แล้วออกแบบ column ให้ตรง
2. **การเชื่อมต่อเว็บกับ Supabase** — ใช้ `supabase.createClient(URL, PublishableKey)` และต้องแยกให้ออกระหว่าง Publishable Key (ใช้ในเว็บได้) กับ Service Role Key (ห้ามวางในเว็บ)
3. **การบันทึกข้อมูล (insert())** — ทุกครั้งที่เรียก `insert()` จะเพิ่ม row ใหม่ ต่างจาก `setItem()` ที่เขียนทับค่าของเดิม
4. **การโหลดข้อมูล (select())** — `select("*")` ขอดึงข้อมูลทุก column, `order()` ใช้เรียงลำดับ, และ `map()` ใช้สร้าง HTML จาก array ของข้อมูล
5. **RLS (Row Level Security)** — ถึงจะเชื่อมต่อได้แล้ว ข้อมูลจะอ่าน/เขียนได้ก็ต่อเมื่อ มี RLS policy อนุญาต ไม่งั้นจะเจอ error เรื่อง row-level security policy
6. **การทดสอบข้าม Browser** — ข้อดีของ cloud database คือข้อมูลถูกเก็บไว้ที่เดียว ทุกอุปกรณ์เห็นข้อมูลชุดเดียวกัน ไม่จำกัดเฉพาะเครื่องที่บันทึก

ขอบเขตของบทนี้: เรียน Create + Read — ยังไม่เรียน Update, Delete, Authentication (การลบ row ที่เลือกจะอยู่ในบทถัดไป: `id → ปุ่มลบ → .delete().eq("id", id)`)